1. **Find 10 different model providers and their models released in last 12 months**

Ans: -

Provider	Model	Release Date	Knowledge Cutoff	Token Cost (Input / Output)	Context Window	Public or Private	Best Use Cases
Open AI	GPT-4.1	Apr 14, 2025	Jun 2024	$2 / $8 per 1M tokens	~1M tokens	Private	Coding, agents, long-context enterprise workflows
Anthropic	Claude Sonnet 4	May 2025	Mar 2025	$3 / $15 per 1M	200K tokens	Private	Coding assistants, enterprise reasoning, document analysis
Google DeepMind	Gemini 2.5 Pro	2025	Early 2025	Varies by API tier	~1M tokens	Private	Multimodal AI, video/image understanding, research
xAI	Grok 3	Feb 2025	Not fully disclosed	~$3 / $15 per 1M	131K tokens	Private	Real-time web/X analysis, reasoning, conversational AI
Meta AI	Llama 4 Maverick	Apr 2025	Aug 2024	Open weights	1M tokens	Public/Open Weights	Local deployment, fine-tuning, enterprise private hosting
Meta AI	Llama 4 Scout	Apr 2025	Aug 2024	Open weights	10M tokens	Public/Open Weights	Ultra-long context retrieval and lightweight deployment
Mistral AI	Mistral Medium 3	2025	2024	Lower-cost commercial pricing	128K tokens	Private	Efficient enterprise inference, European-

Provider	Model	Release Date	Knowledge Cutoff	Token Cost (Input / Output)	Context Window	Public or Private	Best Use Cases
							language workloads
DeepSeek	DeepSeek-V3	Jan 2025	Late 2024	~$0.27 per 1M input	128K tokens	Public/Open Weights	Cost-efficient coding and reasoning
Cohere	Command A	Mar 2025	2024	Enterprise pricing	256K tokens	Private	Retrieval-augmented generation (RAG), enterprise search
AI21 Labs	Jamba 1.6	2025	2024	API pricing varies	256K+ tokens	Hybrid/Open	Long-document summarization and enterprise workflows

2. **Read 2-3 articles about how transformer works?**

Ans: - 1. How Transformers Work – AWS Article Summary aws.amazon.com

Core idea: Transformers process entire sequences at once using attention, not step-by-step like RNNs/LSTMs. That makes them fast + great at long-range context. medium.com. The model has transformer blocks stacked together. Each block = 2 main parts: Multi-head self-attention Position-wise feed-forward neural network aws.amazon.com. Vector Embedding

What: Convert words → numbers that capture meaning medium. comSteps: Tokenization: "The weather is pleasant" → ["The", "weather", "is", "pleasant”] Encoding: Each token → integer: "weather" → 305Embedding: Integer → dense vector: "weather" → [0.9, 0.3, 0.2, ...] medium.com. Similar words get similar vectors. These embeddings form the input matrix X. medium.com Positional encoding: Since transformers process all tokens at once, they add position info to each embedding using sine/cosine signals. This preserves word order. aws.amazon.commedium.com

Self-Attention Mechanism

Problem it solves: Word meaning depends on context. “Bank” = financial vs river. medium. comCore idea: Every word looks at every other word and decides how much to "pay attention" to each one. medium.comHow: Uses 3 vectors for each token - Q, K, V: medium.comVectorStands forRoleAnalogyQ = QueryWhat am I looking for?The search requestK = KeyWhat info do I contain?The searchable label/indexV = ValueWhat info should I pass?The stored content/resultmedium.comProcess:Compare each token’s Query with every other

token’s Key → get attention scoresUse scores to mix together the Values medium.comExample: “The cat sat on the mat because it was tired.”

Feed-Forward Network

What: Each transformer block has a "position-wise feed-forward neural network". aws. amazon. comDetails: Applied to each position separately after attention Consists of 2 linear transformations with ReLU activation in between Refines the representation from self-attention medium. comgeeksforgeeks. orgOther parts in each block: Residual connections: Shortcuts that let info skip operationsLayer normalization: Keeps numbers in a stable range for smooth trainingLinear + Softmax: Final layer to make concrete predictions like next word aws. amazon. comd2l.aiFull data flow medium.com

Input Embedding → words to vectorsPositional Encoding → add order infoMulti-Head Attention → words look at each otherFeedforward Layer → process each positionResidual + Layer Norm → stabilizeOutput → encoded context or generated tokens

2. DigitalOcean:

Vector Embedding: "The input embedding is how words or tokens from your text are turned into numbers so the model can understand them

Self-Attention: "First is the multi-head self-attention mechanism, which allows every token in the input to look at every other token, figuring out how important each one is for understanding the current token’s meaning

Feed-Forward: "The second part is a position-wise feed-forward network, which processes each token’s updated representation independently to transform and refine the learned features"

3. The Illustrated Transformer

Vector Embedding: Picture each word like "mouse" turned into a list of numbers [0.1, 0.8, -0.3, ...]. He shows it as little coloured boxes. Similar words get similar colours.

Self-Attention with Q, K, V: "The animal didn't cross the street because it was too tired "He draws it like this: Query: Word "it" asks "what am I referring to? "Keys: Every other word has a label Values: The actual meaning of each word Then he shows coloured lines - "it" has a thick line to "animal", thin line to "street". That's attention. The model learns "it" = "animal".

Feed-Forward: After attention mixes the words together, each word goes through a mini neural network on its own. He calls it "processing" each position. Two linear layers with ReLU in between.